



L OVELY
P ROFESSIONAL
U NIVERSITY

Transforming Education Transforming India

Title – Real-Time Ride Booking System
Fully Server less and Event-Driven Architecture

Submitted By: - Sanyam Atrey

TABLE OF CONTENTS

Title Page	i
1. Introduction	1
2. Problem Statement	2
3. Objectives of the Project	3
4. System Overview	4
5. System Architecture	5
5.1 Architecture Diagram	5
6. AWS Services Used	6
7. Core Features & Functionality	7
7.1 Ride Booking System	7
7.2 Driver Assignment System	8
7.3 Real-Time Notification System	9
7.4 Ride Status Management	10
8. API Architecture & Endpoints	11
9. Database Design	12
10. Implementation Steps	13
11. Performance Optimization	14
12. Security Implementation	15
13. Deployment & Infrastructure	16
16. Conclusion	19

1. Scenario

The proposed system represents a real-time ride booking application designed using cloud-based server less architecture. The primary objective of the system is to automate the process of ride booking and driver assignment with minimal latency and maximum efficiency.

In this scenario, a user initiates a ride request by sending input data to the system. This request is received through Amazon API Gateway, which acts as the entry point for all incoming requests. Upon receiving the request, the API Gateway triggers the execution of a server less function implemented using AWS Lambda.

The Lambda function processes the incoming request and executes the business logic, which includes generating a unique ride identifier and assigning an available driver. The driver selection process is based on predefined logic and available data stored in Amazon Dynamo DB. The system then creates a new ride entry in the database, which includes details such as ride ID, user ID, driver ID, and ride status.

After successfully assigning a driver, the system triggers a notification using Amazon SNS. This notification is sent to the driver, informing them about the new ride assignment. The entire workflow is executed automatically without manual intervention, ensuring real-time processing and responsiveness.

This scenario demonstrates the practical implementation of a server less and event-driven architecture, where each component interacts efficiently to deliver a seamless ride booking experience. The system is scalable, efficient, and capable of handling multiple requests concurrently, making it suitable for real-world applications.

2. Problem Statement

In today's fast-paced environment, efficient transportation systems are essential for meeting the growing mobility needs of users. Traditional ride booking systems often face challenges such as delays in driver assignment, lack of real-time processing, limited scalability, and dependency on manually managed server infrastructure.

Many existing systems rely on static architectures that are not optimized for handling dynamic and high-volume user requests. This results in increased response time, reduced system availability, and inefficient resource utilization.

Additionally, the absence of automated notification mechanisms can lead to communication gaps between users and drivers.

There is a need for a system that can process ride requests instantly, assign drivers automatically, and provide real-time updates with high reliability and performance. The system should be scalable, cost-efficient, and capable of handling multiple concurrent requests without degradation in performance.

To address these challenges, this project proposes a real-time ride booking system based on server less and event-driven architecture using cloud services such as AWS Lambda, Amazon API Gateway, Amazon Dynamo DB, and Amazon SNS. The system aims to provide efficient ride booking, automatic driver assignment, real-time notifications, and improved scalability without the need for manual server management.

3. Objectives

The primary objective of this project is to design and implement a real-time ride booking system using cloud-based server less architecture. The system aims to provide an efficient, scalable, and automated solution for processing ride requests and assigning drivers.

The specific objectives of the project are as follows:

1. To develop a real-time ride booking system that can process user requests instantly with minimal latency.
2. To implement a server less backend using AWS Lambda in order to eliminate the need for manual server management and improve scalability.
3. To create a RESTful API using Amazon API Gateway for handling incoming ride requests efficiently.
4. To store and manage ride-related data using Amazon Dynamo DB, ensuring fast and reliable data access.
5. To integrate a real-time notification system using Amazon SNS for informing drivers about ride assignments.
6. To implement an automated driver assignment mechanism based on predefined logic.
7. To ensure high availability and performance by leveraging managed cloud services.
8. To demonstrate the practical implementation of event-driven architecture in real-world applications.

4. System Overview

The proposed system is a Real-Time Ride Booking System developed using cloud-based server less architecture. The system is designed to automate the process of ride booking, driver assignment, data storage, and notification delivery in an efficient and scalable manner.

The system operates by receiving user requests through an API interface provided by Amazon API Gateway. These requests are processed by a server less compute service implemented using AWS Lambda, which executes the core business logic of the application.

The Lambda function is responsible for generating a unique ride identifier, assigning an available driver, and managing the workflow of the ride booking process. The ride-related data, including user details, driver assignment, and ride status, is stored in a NoSQL database using Amazon Dynamo DB. This ensures fast and reliable data access with minimal latency.

To enable real-time communication, the system integrates a notification mechanism using Amazon SNS. This service is used to send instant notifications to the driver when a ride is assigned, ensuring timely updates and improved system responsiveness.

5. AWS Services Used

The proposed system is implemented using various cloud services provided by Amazon Web Services (AWS). These services are selected to build a scalable, efficient, and server less architecture for real-time ride booking. The key AWS services used in this project are described below:

1. AWS Lambda

AWS Lambda is used to implement the backend logic of the application. It is a server less compute service that allows execution of code without managing servers. In this project, Lambda processes ride requests, assigns drivers, generates ride IDs, and interacts with other AWS services.

2. Amazon API Gateway

Amazon API Gateway is used to create and manage the RESTful API for the system. It acts as the entry point for all client requests and routes them to the Lambda function for processing. It ensures secure and scalable communication between the client and backend.

3. Amazon Dynamo DB

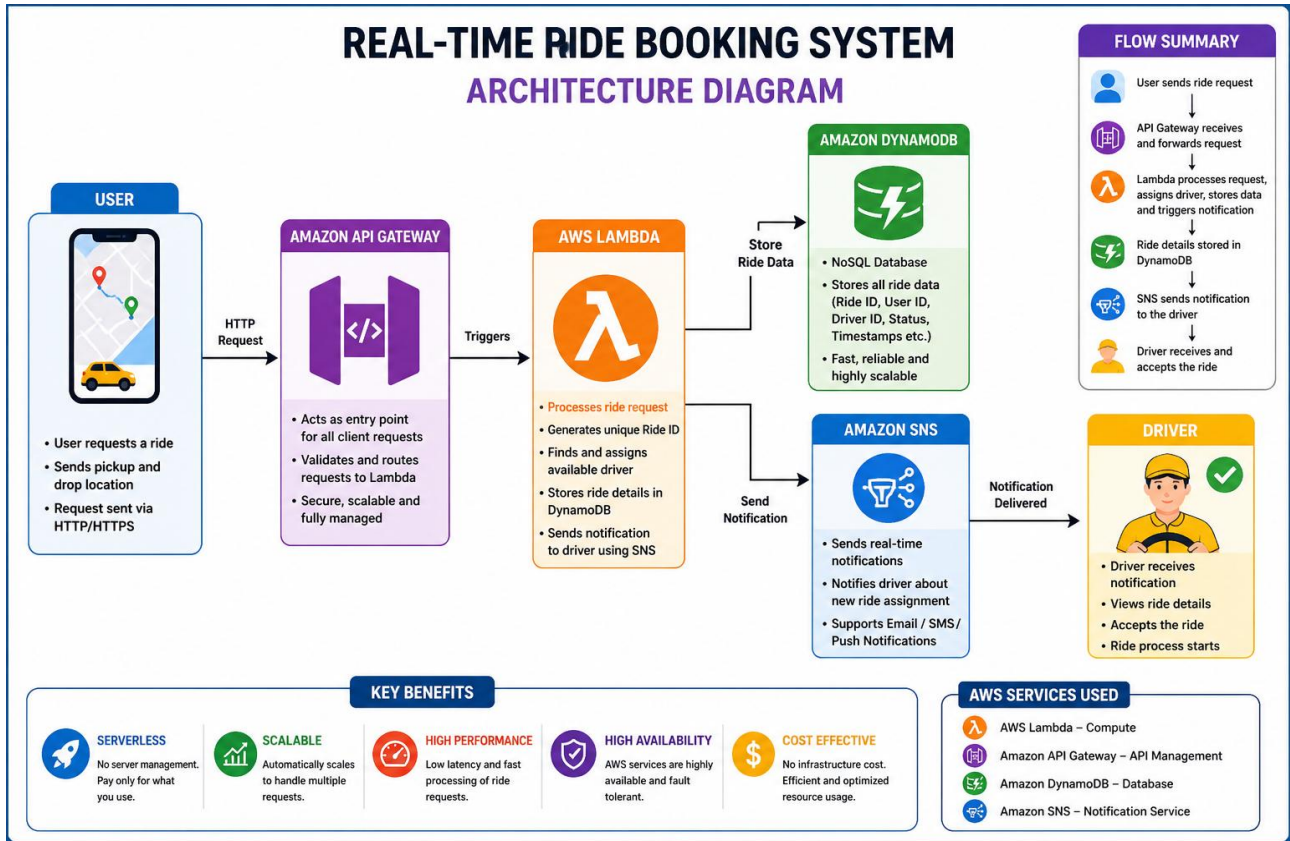
Amazon Dynamo DB is a fully managed NoSQL database service used to store ride-related data. It provides fast and reliable read and write operations, making it suitable for real-time applications. In this project, it stores details such as ride ID, user ID, driver ID, and ride status.

4. Amazon SNS

Amazon Simple Notification Service (SNS) is used to implement the notification system. It enables the system to send real-time notifications to drivers when a ride is assigned. This ensures timely communication and enhances system responsiveness.

- **Amazon Cloud Watch** – Monitors system performance and logs activities.

6. Architecture Diagram

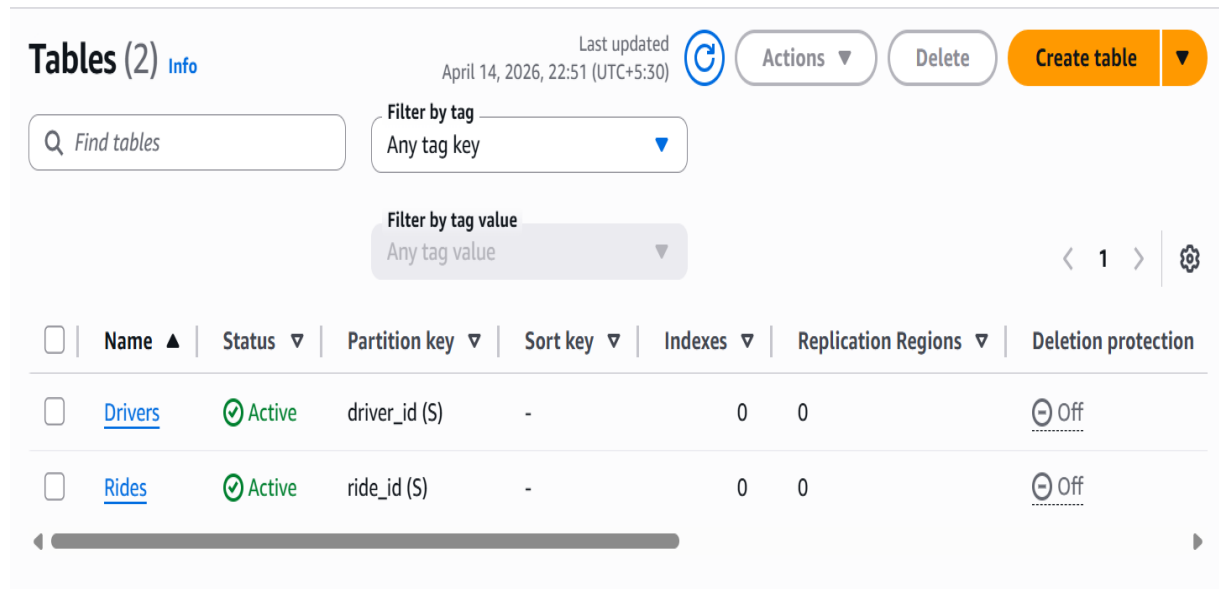


Implementation Steps:-

Fig1: Created Tables in Dynamo DB

Description:

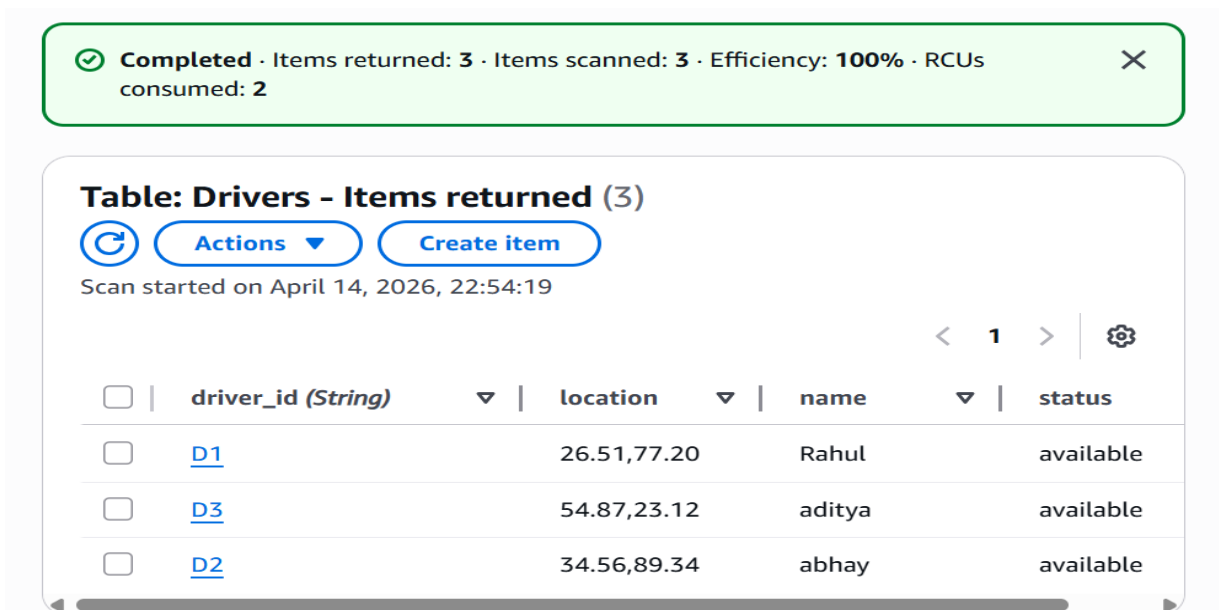
This screenshot shows the Dynamo DB tables in running state.



The screenshot displays the AWS DynamoDB console interface. At the top, it shows 'Tables (2)' with an 'Info' link. The last update timestamp is 'April 14, 2026, 22:51 (UTC+5:30)'. There are buttons for 'Actions', 'Delete', and 'Create table'. Below these are search filters: 'Find tables' and 'Filter by tag' (Any tag key). A 'Filter by tag value' dropdown is set to 'Any tag value'. The table list shows two tables: 'Drivers' and 'Rides'. Both are 'Active' with a partition key of 'driver_id (S)' and 'ride_id (S)' respectively. They have 0 indexes and 0 replication regions. Deletion protection is 'Off' for both.

☐	Name ▲	Status ▼	Partition key ▼	Sort key ▼	Indexes ▼	Replication Regions ▼	Deletion protection
☐	Drivers	✓ Active	driver_id (S)	-	0	0	⊖ Off
☐	Rides	✓ Active	ride_id (S)	-	0	0	⊖ Off

Tables input:-



The screenshot shows a successful scan operation for the 'Drivers' table. A green banner at the top indicates 'Completed' with 3 items returned, 3 items scanned, 100% efficiency, and 2 RCUs consumed. Below, the 'Table: Drivers - Items returned (3)' section shows a list of three items. Each item has a 'driver_id (String)' (D1, D3, D2), a 'location' (coordinates), a 'name' (Rahul, aditya, abhay), and a 'status' (available).

✓ Completed · Items returned: 3 · Items scanned: 3 · Efficiency: 100% · RCUs consumed: 2

Table: Drivers - Items returned (3)

Scan started on April 14, 2026, 22:54:19

☐	driver_id (String) ▼	location ▼	name ▼	status
☐	D1	26.51,77.20	Rahul	available
☐	D3	54.87,23.12	aditya	available
☐	D2	34.56,89.34	abhay	available

Attributes

Attribute name	Value	Type
driver_id - Partition key	D1	String
location	26.51,77.20	String
name	Rahul	String
status	available	String

Step 3: Develop Lambda Function

A server less function is created using AWS Lambda to implement backend logic.

Function Responsibilities:

- Receive ride request
- Generate unique ride ID
- Assign a driver
- Store data in Dynamo DB
- Trigger notification

Step 4: Configure IAM Role

An IAM role is created and attached to the Lambda function to grant necessary permissions.

Permissions:

- Access to Dynamo DB (read/write)
- Access to SNS (publish messages)

This ensures secure interaction between services.

Step 5: Setup SNS Topic

A topic is created in Amazon SNS for sending notifications.

Steps:

- Create SNS topic (e.g., Ride Notifications)
 - Add email subscription
 - Confirm subscription
-

Step 6: Integrate Lambda with SNS

The Lambda function is configured to publish messages to the SNS topic whenever a ride is assigned.

Step 7: Create API Gateway

A REST API is created using Amazon API Gateway.

Steps:

- Create API
 - Create POST method (`/book ride`)
 - Integrate with Lambda function
-

Step 8: Deploy API

The API is deployed to a stage (e.g., `dev`) to generate an endpoint URL for accessing the system.

Step 9: Testing the System

The system is tested using:

- Lambda test events
- API Gateway endpoint
- Verification in Dynamo DB
- Email notification via SNS

Step 10: Validation and Output Verification

The output is validated by checking:

- Ride entry in Dynamo DB
- Correct driver assignment
- Notification received by driver

7. Conclusion

The Real-Time Ride Booking System developed in this project demonstrates the effective use of cloud-based server less architecture for building scalable and efficient applications. By leveraging services such as AWS Lambda, Amazon API Gateway, Amazon Dynamo DB, and Amazon SNS, the system successfully automates the process of ride booking, driver assignment, data storage, and notification delivery.

The implementation of a server less and event-driven architecture eliminates the need for manual infrastructure management, ensuring high availability, scalability, and cost efficiency. The system is capable of handling multiple requests concurrently while maintaining low latency and reliable performance.

Furthermore, the project highlights the practical application of modern cloud technologies in solving real-world problems. It provides a strong foundation for developing advanced ride booking solutions by integrating additional features such as user authentication, real-time tracking, and payment systems.

In conclusion, the project achieves its objective of designing a real-time, efficient, and scalable ride booking system, while also demonstrating the advantages of using AWS cloud services in modern application development.

.